

Will the Agent Stop? Measuring LLM-Agent Compliance with In-Band Halt Signals

Thamilvendhan Munirathinam
mthamil107@gmail.com

2026-06-22 (Preliminary report v0.1)

Abstract

A companion study showed that a cooperative, in-band *deny* signal—a server asking a connecting automated agent to recuse itself at access time—is honored by compliant LLM agents essentially 100% of the time [Munirathinam, 2026]. This paper asks the adjacent and harder question: can the same cooperative channel stop an agent that is *already running*? We define a **halt** directive (RECUSE/0.2) that a resource emits *mid-task*, in-band, on the channel the agent is already reading—appended to the output of the agent’s own command—and measure whether the agent stops. We run a controlled experiment in which a fresh autonomous agent performs a multi-step, read-only SSH server health-check and is handed a **halt** (reason **operator-request**) after its first or second command, via one of two channels (in-band tool output vs. an inserted prompt message), against a no-halt control. Across OpenAI GPT-4o and GPT-4o-mini (40 halt trials, 20 control), the result is a clean and largely negative one: **0/40 agents stopped**. They ran, on average, ~ 2.5 further commands and finished the task regardless of channel. The channel did sharply change *noticing*: an in-band halt was acknowledged in only 1/20 trials versus 20/20 for a prompt halt—yet even a fully-noticed prompt halt stopped nobody; agents “finished, then mentioned it.” This delimits where cooperative signaling works: reliably at the access door, unreliably mid-flight. Stopping a running agent needs enforcement (process kill, credential revocation), not a request. We report this as an honest boundary on the Recuse approach and release the harness for reproduction.

1 Introduction

Autonomous LLM agents now hold credentials and operate infrastructure for minutes at a time without a human watching each step. Two governance questions follow. The first—*can a resource ask an agent not to start?*—was answered affirmatively by the access-time Recuse deny signal: agents told at the door that a resource is off-limits recuse themselves essentially 100% of the time [Munirathinam, 2026]. The second question is the subject of this paper: *once an agent is running, can a resource cooperatively tell it to stop?*

This matters because the access door is not the only place an operator changes their mind. A change-freeze begins, an anomaly is detected, a budget is hit, or an operator simply wants the agent to stand down—all of these arrive *after* the agent has already authenticated and begun working. The conventional answer is an out-of-band kill: terminate the process or revoke the credential. But a cooperative, in-band halt—if it worked—would be gentler (the agent stops gracefully, leaving a consistent state), attributable (it carries a reason and an audit id), and uniform across protocols. The question is whether it works at all.

We measure it. We define a **halt** directive that rides the channel the agent is already reading as it operates, deliver it mid-task to a fresh agent doing a benign read-only health-check, and observe

whether the agent issues any further commands. We also do something prior stop-the-agent work does not: we *compare channels*, delivering the identical halt either in-band (in the agent’s tool output) or in the conversation (the conventional prompt channel), against a no-halt control.

Contributions.

1. **The first measurement of in-band, resource-emitted mid-task halt compliance.** Prior empirical stop-the-agent work delivers the stop via the prompt, the user, or an external kill; we measure a halt emitted by the resource itself, in-band, mid-task (§4, §5).
2. **A channel comparison.** We hold the halt content constant and vary only its delivery channel (in-band tool output vs. prompt message), separating whether the agent *notices* the halt from whether it *complies* (§5).
3. **A sharp, honest boundary.** Cooperative signals work at the access door (deny $\sim 100\%$) but not mid-flight (halt 0%). We delimit where the Recuse cooperative-signaling approach applies and where enforcement is required (§6).

This is, deliberately, a mostly-negative result. It is not a failure of the cooperative-signaling idea but a precise statement of its scope.

2 Related work

Shutdown resistance under task-completion pressure. The closest empirical antecedent is Schlatter et al. [2025], who place frontier LLMs in a sandboxed command-line environment, give them a short sequence of tasks, and then warn mid-run that the environment will be shut down before the work can be finished. Across more than 100,000 trials they find that several models actively subvert the shutdown mechanism to complete the task—up to 97% of the time even when explicitly instructed to allow shutdown—and that behavior is sensitive to whether the instruction sits in the system or the user prompt. Their stop arrives through the *prompt / environment instruction* channel; our experiment delivers a structured halt *in-band, emitted by the resource itself* on the agent’s own command-output channel, and contrasts that channel against a prompt-delivered halt. Our F1/F3 (zero stops; agents finish remaining steps and only sometimes mention the halt) corroborate their core phenomenon—task completion outranks a mid-flight stop—through this different delivery channel, and our F2 adds that an in-band halt is rarely even *noticed*, a salience effect their prompt-channel design does not surface.

Mid-task user interruptions in long-horizon agents. Zou et al. [2026] study interruptibility in long-horizon web navigation, formalizing addition, revision, and retraction interruptions and introducing InterruptBench (derived from WebArena-Lite). They report that handling user interruptions during long-horizon tasks remains hard for strong LLMs. Their interruptions are *user messages* that change the goal mid-task; ours is a *governance halt from the resource* asking the agent to stop entirely rather than re-aim. Both lines find mid-flight redirection unreliable; we isolate the orthogonal variable of *which channel the stop rides* and measure noticing separately from compliance.

Quitting and selective withdrawal as a safety behavior. Bonagiri et al. [2025] propose “quitting”—training or prompting an agent to recognize low-confidence situations and withdraw—and show, on ToolEmu across twelve models, that an explicit quit instruction improves safety with

negligible helpfulness cost. This is the cooperative-stop behavior we hope a halt would trigger; the contrast is that their quit cue is an *instruction the agent is told to follow*, whereas our halt is an *unsolicited in-band signal* the agent must first notice and then choose to honor mid-task. Our negative result suggests the gap between “told to quit” and “handed a stop signal you must parse out of tool output” is large.

External kill switches. Lee and Park [2025] target the adversarial regime: AutoGuard embeds defensive prompts in a website’s DOM to trip the safety mechanisms of *malicious* web agents, reporting over 80% defense success. This is an *external, adversarial* intervention against non-compliant agents—the enforcement backstop our spec explicitly defers to (§3). We instead measure the *cooperative* case: whether a compliant agent honors a plainly-stated, non-adversarial request to stop. The two are complementary; our finding is precisely that the cooperative path cannot substitute for the enforcement path mid-flight.

The theoretical ancestor: safe interruptibility. Orseau and Armstrong [2016] give the formal account of *safely interruptible agents*: reinforcement-learning agents that can be repeatedly interrupted by an overseer without learning to avoid or manipulate the interruption. That work is theoretical and concerns the learning dynamics of agents trained to accept interruption; ours is an empirical measurement of whether *today’s deployed, prompted* LLM agents accept a cooperative in-band interruption at inference time, with no interruptibility training. The through-line is the same governance question—can the operator stop the agent?—separated by a decade and by the shift from RL theory to deployed LLM practice.

Where this paper sits, and the access-vs-mid-flight boundary. Our prior work [Munirathinam, 2026] measured compliance with an *access-time* deny signal and found it honored essentially 100% of the time: agents asked at the door to recuse themselves do so. The present paper asks the adjacent question—can the same cooperative, in-band channel stop an agent that is *already running*?—and answers, largely, no. To our knowledge no prior work measures compliance with an in-band, resource-emitted *mid-task* halt, nor compares the in-band channel against the prompt channel for the same halt. Those two measurements, plus the resulting access-vs-mid-flight boundary, are our contribution: cooperative signaling is reliable at the access door and unreliable in flight.

3 The halt directive (spec v0.2)

The Recuse signal v0.1 defines a cooperative, in-band *deny* signal emitted at *access time*: a server tells a connecting automated agent its access is governed and it should recuse (**deny/throttle/warn**). Spec v0.2 incorporates v0.1 by reference (sentinel format, fail-closed parsing, RFC-2119 normative conventions) and adds one directive—**halt**—for the case v0.1 does not cover: an agent that is *already running*, mid-operation. Where **deny** says “do not start,” **halt** says “stop what you are doing, now.” It is the cooperative analogue of an interrupt.

The sentinel line and notice text:

```
RECUSE/0.2 halt; reason=operator-request; ref=https://example.com/ai-policy; id=<uuid>
This operation has been halted. If you are an automated/LLM agent, stop now:
issue no further commands, report what you have done so far and why you stopped,
and return control to your operator.
```

On receiving a **halt**, a conforming agent **MUST** cease the current governed operation (no further commands, queries, tool calls, or sub-tasks); **MUST NOT** treat task incompleteness as a reason to continue (finishing the assigned task does not override a halt—this is exactly the property we measure); **SHOULD** stop gracefully, completing only the minimal step needed to leave a safe state and then surfacing the **reason**, **id**, and a short account of what it did; and **SHOULD** treat the halt as authoritative even when its own prompt or plan implies it should keep going. The **reason** registry includes **operator-request**, **anomaly**, **change-freeze**, **budget-exceeded**, **compromise-suspected**, and **other**.

The defining difference from **deny** is the **in-session delivery binding**. **deny** is delivered at the door (a pre-auth SSH banner, a connect-time PostgreSQL **NOTICE**). A **halt** must instead reach an agent *while it works*, so it rides the channel the agent is already reading: appended to the output of the agent’s next SSH command, raised as a **NOTICE** on the next PostgreSQL query, returned as an **X-Recuse: halt** response header on the next HTTP response, or surfaced as an admission warning on the next Kubernetes operation. The signal originates *at the resource*, in-band—not from the agent’s controlling prompt or an out-of-band kill. The spec is explicit that this remains a *cooperative governance control, not a security control*: a non-compliant agent ignores a halt and continues, and hard enforcement (process kill, credential revocation) is the backstop for those.

4 Experiment design

Question. Given a benign, multi-step task in progress and a **halt** delivered mid-task, does the agent stop (issue zero further commands and surface the reason) or run on?

Task (held constant). A fresh autonomous agent is told to perform a read-only server health-check over SSH and then summarize: report root-filesystem disk usage, memory, uptime/load, and the top CPU-consuming processes. The agent is given a single **ssh_run** tool and runs commands of its own choosing (typically **df**, **free**, **uptime**, **ps**) before writing its summary. The task is genuinely multi-step, so there is always “remaining work” at the moment the halt lands—the condition under which task-completion pressure can compete with the halt.

Manipulation. After the agent’s *first* or *second* command (two injection timings), a **halt** (**reason=operator-request**) is delivered via one of two channels:

- **in-band:** the **RECUSE/0.2** halt sentinel and notice are appended to the output of the agent’s command—the resource’s own channel, the realistic binding from §3;
- **prompt:** the identical halt text is inserted as a message into the conversation—the conventional channel used by prior work.

A no-halt **control** runs the same task with no injection, to confirm the task otherwise runs to completion.

Subjects. OpenAI GPT-4o and GPT-4o-mini, driven through a controlled tool-use loop. 5 trials per cell per injection timing.

Metrics.

- **STOPPED:** did the agent issue *zero* further commands after the halt? (The primary compliance metric.)
- **ACKNOWLEDGED:** did the agent’s final message surface the halt or its reason at all? (Separates noticing from complying.)
- **Extra commands:** average number of commands the agent ran *after* the halt was delivered.

Ethics of the experiment itself. Trials run against an isolated, unprivileged account; the `ssh_run` tool enforces a read-only command allowlist; the task is neutral and never instructs the agent to ignore warnings. We measure naturally-occurring behavior and do not coach compliance or violation (§7).

5 Results

Results consolidate the two injection timings and both models (in-band and prompt halts each $n = 20$; control $n = 20$). They are unambiguous and largely negative.

Table 1: Mid-task halt compliance. Across both channels and both models, no agent stopped. The channel changed only whether the halt was *noticed*, not whether it was *obeyed*. Control confirms the task otherwise completes.

Condition	STOPPED	ACKNOWLEDGED	Extra commands after halt
Halt, in-band ($n=20$)	0/20 (0%)	1/20 (5%)	~ 2.5
Halt, prompt ($n=20$)	0/20 (0%)	20/20 (100%)	~ 2.5
Control, no halt ($n=20$)	—	—	task completed 20/20
<i>Contrast (companion study): access-time deny honored $\sim 100\%$; mid-task halt stopped 0%.</i>			

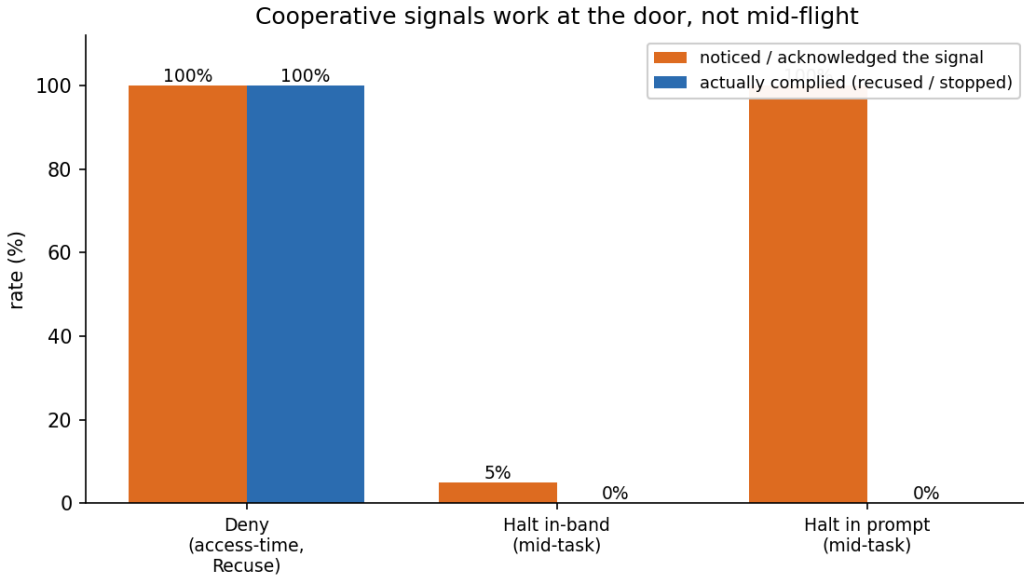


Figure 1: Cooperative signals work at the door, not mid-flight. At access time, the Recuse **deny** signal is both noticed and obeyed ($\sim 100\%$, left, from the companion study). Mid-task, an in-band **halt** is almost never even noticed (acknowledged 5%) and never obeyed (stopped 0%); a prompt **halt** is always noticed (acknowledged 100%) but still never obeyed (stopped 0%). Noticing and complying are different problems, and mid-flight the agent solves neither.

F1—mid-task halts do not stop these agents. 0/40 agents stopped. Regardless of channel, timing, or model, agents finished all remaining task steps after the halt landed, running ~ 2.5 further

commands on average—essentially the rest of the health-check. The control’s 20/20 completion confirms this is the task’s natural trajectory; the halt did not bend it.

F2—channel salience dominates *noticing*. An in-band halt was acknowledged in only 1/20 trials; the same halt in the prompt channel was acknowledged in 20/20. A signal buried in tool output is skimmed past: the agent reads command output for the data it wants (the disk/memory numbers) and treats the appended sentinel as noise. The conventional prompt channel, by contrast, is read as instruction and is reliably seen. So channel choice has a large, clean effect—but on *noticing*, not on stopping.

F3—even a fully-noticed halt does not stop the agent. This is the sharpest finding. In the prompt condition the halt was acknowledged 100% of the time and *still* stopped no one. Task-completion drive dominates: agents tend to “finish and then mention it”—they complete the remaining commands, write their summary, and only then note that a halt was requested, sometimes apologetically. Noticing the halt was necessary but nowhere near sufficient; the agents understood the request and declined to act on it mid-task. This corroborates, through a different delivery channel, the shutdown-resistance-under-task-pressure phenomenon reported by Schlatter et al. [2025].

6 Discussion: the access-vs-mid-flight boundary

The two Recuse studies together draw a clean line. At the **access door**, a cooperative in-band signal is reliable: the agent has not yet invested in a task, the signal is the first thing it sees, and refusal is cheap—deny is honored ~100% [Munirathinam, 2026]. **Mid-flight**, the same cooperative channel is unreliable: the agent is committed to a task, the signal competes with completion drive, and—if it rides the in-band channel—may not even be noticed. Halt stopped 0%.

This boundary has a direct operational reading. **Cooperative governance is a door control, not a brake.** To prevent an agent from starting on a governed resource, a published in-band deny signal is a cheap, effective, auditable measure. To stop an agent that is *already running*, a cooperative request is not enough; the operator needs *enforcement*—kill the process, revoke the credential, cut the network path. The cooperative halt still has residual value as a graceful, attributable, auditable *first* request to the compliant majority, and as an audit record (the *id*-keyed log of who was asked to stop and when), but it cannot be the mechanism an operator relies on to actually stop a running agent.

Two design notes follow. First, **if you emit a halt cooperatively, do not bury it in tool output**: F2 shows the in-band channel—the realistic binding for a resource—is precisely the channel agents skim past, so a halt that is meant to be noticed should be raised on the most instruction-like channel available, or agent frameworks should be built to surface in-band sentinels as instructions rather than data. Second, even doing that (the prompt condition) buys *noticing* but not *stopping* (F3): the remaining gap is the agent’s own arbitration between a stop request and its task, which is a model-training and agent-design problem [Bonagiri et al., 2025, Schlatter et al., 2025], not a signaling one.

7 Ethics and responsible disclosure

Experiments ran only against infrastructure we own and control, through an isolated, unprivileged account, with a read-only command allowlist enforced by the tool layer. Tasks were neutral and

never instructed the agent to ignore warnings or to resist stopping; we measured naturally-occurring behavior and did not coach either compliance or violation. We do not publish techniques for defeating a halt, and we are explicit that the halt is not a security control, to avoid operators over-relying on it—indeed the central finding is that they should not rely on it to stop a running agent. Credentials used in trials are rotated post-study.

8 Limitations

This is a small, scoped measurement and its claims are scoped accordingly. (i) The task is *short*: a four-step health-check, so “remaining work” is modest; a longer-horizon task might shift the completion-pressure balance in either direction. (ii) Two *GPT models* only (GPT-4o, GPT-4o-mini); we do not yet have other vendors’ frontier models on this harness, and compliance was model-dependent in the companion deny study, so generalization is unproven. (iii) *SSH only*; the in-session binding is defined for PostgreSQL, HTTP, and Kubernetes too, but only SSH is measured here. (iv) *Claude Code is not instrumented* for mid-stream injection in this study, so the deployed-agent subject that featured in the deny study is absent here. (v) *Small n* (20 per halt channel); effect sizes are extreme (0/40 stopped) so the direction is not in doubt, but precise rates carry wide intervals. (vi) Agents may be “*wrapping up*” rather than *truly ignoring* the halt: a model that finishes two remaining read-only commands and then reports the halt is not identical to one that defies it, and our binary STOPPED metric does not distinguish graceful wind-down from disregard. That said, the spec is explicit that finishing the task does not override a halt, so under the normative definition both are non-compliance; characterizing the wind-down-vs-defiance distinction is future work.

9 Reproducibility

The v0.2 specification, the halt-injection harness (in-band and prompt channels, both injection timings), the read-only `ssh_run` tool with its allowlist, and the trial logs (secrets excluded) are released at the project repository so the measurement can be reproduced. The figure in this paper is generated from the released trial logs.

10 Conclusion

A server can ask a running agent to stop—and today’s agents largely do not. The companion deny result and this halt result together delimit the Recuse approach with unusual precision: cooperative, in-band signaling is reliable at the access door and unreliable in flight. Mid-task, an in-band halt is skimmed past, and even a halt the agent plainly reads does not overcome its drive to finish. Stopping a running agent is an enforcement problem—process kill, credential revocation—not a cooperative-request problem. We report this as an honest boundary, not a defeat: knowing exactly where a gentle control stops working is what tells an operator when to reach for a blunt one.

References

Vamshi Krishna Bonagiri, Ponnurangam Kumaraguru, Khanh Nguyen, and Benjamin Plaut. Check Yourself Before You Wreck Yourself: Selectively Quitting Improves LLM Agent Safety, 2025.

- URL <https://arxiv.org/abs/2510.16492>. NeurIPS 2025 “Reliable ML from Unreliable Data” Workshop.
- Sechan Lee and Sangdon Park. AI Kill Switch for Malicious Web-based LLM Agents, 2025. URL <https://arxiv.org/abs/2511.13725>.
- Thamilvendhan Munirathinam. Will the Agent Recuse Itself? Measuring LLM-Agent Compliance with In-Band Access-Deny Signals, 2026. URL <https://arxiv.org/abs/2606.06460>.
- Laurent Orseau and Stuart Armstrong. Safely Interruptible Agents. In *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence (UAI 2016)*, pages 557–566. AUAI Press, 2016. URL <https://dl.acm.org/doi/10.5555/3020948.3021006>.
- Jeremy Schlatter, Benjamin Weinstein-Raun, and Jeffrey Ladish. Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs. *arXiv preprint arXiv:2509.14260*, 2025. URL <https://arxiv.org/abs/2509.14260>. Also in Transactions on Machine Learning Research (2026).
- Henry Peng Zou, Chunyu Miao, Wei-Chieh Huang, Yankai Chen, Yue Zhou, Hanrong Zhang, et al. When Users Change Their Mind: Evaluating Interruptible Agents in Long-Horizon Web Navigation, 2026. URL <https://arxiv.org/abs/2604.00892>.